

■ Guia Técnico — DRE - DRPS

Versão: 8.0

Atualizado: 12/06/2026

Público: Equipe Técnica / Desenvolvedores

■ Índice

1. Arquitetura (#arquitetura)
2. Estrutura de Dados (Supabase) (#estrutura-de-dados)
3. Metodologia de Grau de Risco (GR) v2 (#metodologia-de-grau-de-risco-gr-v2)
4. Fluxos de Dados (#fluxos-de-dados)
5. Configuração e Segredos (#configuração-e-segredos)
6. Camada de Banco — db.py (#camada-de-banco--dbpy)
7. Segurança (#segurança)
8. Troubleshooting Técnico (#troubleshooting-técnico)
9. Dependências (#dependências)
10. Próximas Melhorias (#próximas-melhorias)

■ ■ Arquitetura

Stack Tecnológico

...

[illegible]

■ Frontend (Navegador) ■

■ HTML5 + JS (renderizado pelo Streamlit) ■

[illegible]

■ HTTPS

V

■ Streamlit Cloud — app.py (Python) ■

■ • app.py — 5 módulos (Início / Questionários / RH / ■

■ Documentação / Admin), ~3.200 linhas ■

■ • db.py — camada de acesso ao Supabase ■

■ • gerar laudo.py — laudo DRPS (PDF, ReportLab) ■

■ • gerar_laudo_aep.py — laudo DRE (PDF, ReportLab) ■

■ • `.streamlit/config.toml` — tema visual ■

[illegible]

- supabase-py (API REST)

[illegible]

■ Supabase (PostgreSQL) ■

■ Tabelas: acessos, respostas, respostas_aep, usuarios, ■

■ laudos, config ■

[illegible]

...

Componentes Principais

1. app.py / app_cloud.py (núcleo)

- Define o menu lateral (`_opcoes_menu`) e despacha para 5 módulos via

```
if/elif menu == "...".
```

- Mantém estado de sessão (`st.session_state`) para login (Admin/RH), navegação entre blocos do questionário e modo demonstração.
- `app_cloud.py` é a versão publicada no Streamlit Cloud (idêntica a `app.py` em funcionalidade).

2. db.py (acesso ao Supabase)

- Toda a persistência passa por aqui — **não há mais CSVs locais**.
- Usa `st.cache_resource` para reaproveitar o cliente Supabase (`_get_sb()`), criado a partir de `st.secrets["SUPABASE_URL"]` e `st.secrets["SUPABASE_KEY"]`.
- `ping()` é chamado no início do app para "despertar" o banco (plano gratuito do Supabase hiberna após 7 dias sem uso).
- Faz a tradução entre nomes de coluna "amigáveis" (ex.: CPF, Empresa) usados no app e os nomes em `snake_case` das tabelas (ex.: `cpf`, `empresa`).

3. gerar_laudo.py / gerar_laudo_aep.py (geração de PDF)

- `gerar_laudo.py` → laudo **DRPS** (Psicossocial / COPSOQ III), 10 seções.
- `gerar_laudo_aep.py` → laudo **DRE** (Ergonômico / AEP / NR-17), 8 seções, incluindo o inventário de riscos com o Grau de Risco (GR) v2.
- Ambos usam ReportLab (`platypus`, `Table`, `Paragraph`, etc.).

4. .streamlit/config.toml (tema visual)

```

toml

[theme]
primaryColor = "#DC3B24"
backgroundColor = "#EFEFEF"
secondaryBackgroundColor = "#FFFFFF"
textColor = "#282C5B"
font = "sans serif"

```

■ Estrutura de Dados (Supabase)

Todas as tabelas vivem em um projeto Supabase (PostgreSQL). `db.py` faz a conversão entre os nomes usados no app (CamelCase) e as colunas no banco (snake_case).

Tabela `acessos` (cadastro de empresas e colaboradores)

Coluna (banco)	Nome no app	Descrição
cpf	CPF	11 dígitos, sem formatação
empresa	Empresa	Razão social
cnpj	CNPJ	14 dígitos, sem formatação
funcao	Função	Cargo/posição
departamento	Departamento	Setor
cnae	CNAE	CNAE principal da empresa
grau_risco	Grau de Risco (NR-4)	"1" a "4" — usado na calibração de severidades do DRE
data_acesso_liberado	Data_Acesso_Liberado	Data do cadastro
data_inicio_periodo	Data_Inicio_Periodo	Início da janela de resposta
data_fim_periodo	Data_Fim_Periodo	Fim da janela de resposta
status	Status	"Ativo" / "Inativo"
data_movimentacao	Data_Movimentacao	Data da última ativação/inativação
motivo_movimentacao	Motivo_Movimentacao	Texto livre (opcional)

Tabela `respostas` (DRPS — Psicossocial)

Coluna	Descrição
cpf_hash	SHA-256(CPF) — identificador anônimo
cnpj, empresa, funcao, departamento	Metadados da resposta
data	Data/hora da resposta ("dd/mm/aaaa HH:MM")
media_geral	Média geral COPSOQ III
media_*	Média por dimensão (cargo, controle, demandas, etc., já com inversão aplicada)

Tabela `respostas\_aep` (DRE — Ergonômico)

Coluna	Descrição
-----	-----
cpf_hash	SHA-256(CPF)
cnpj, empresa, departamento	Metadados
data	Data da resposta ("aaaa-mm-dd")
q1...q17	Respostas às 17 perguntas (seções A-D)
relato_dor, relato_sugestoes	Campos abertos de texto
relato_dificuldades	Campo legado, mantido para retrocompatibilidade

- > ■■ As tabelas respostas e respostas_aep armazenam datas em formatos > diferentes ("dd/mm/aaaa HH:MM" vs. "aaaa-mm-dd"). Ao processar ambas, o > parsing de data deve ser feito **por formato explícito** — usar > dayfirst/format="mixed" interpreta o formato ISO incorretamente.

Tabela `usuarios` (login da equipe SSTG — perfil operacional)

Coluna	Descrição
-----	-----
usuario	Login
nome	Nome de exibição
senha_hash	Hash da senha
status	"Ativo" / "Inativo"
data_criacao	Data de criação do usuário

Tabela `laudos` (histórico de laudos gerados)

Coluna	Descrição
-----	-----
id	Identificador
data	Data de geração
cnpj	Empresa
tipo	"DRPS" ou "DRE"

- > Esta tabela precisa existir no Supabase para que o painel ■ **Início**

> mostre o indicador "Laudos gerados". As funções registrar_laudo() e
> contar_laudos() falham silenciosamente se a tabela não existir.

Tabela `config` (parâmetros gerais)

Coluna	Descrição
-----	-----
chave	Nome do parâmetro (ex.: senha_admin_hash)
valor	Valor armazenado

■ Metodologia de Grau de Risco (GR) v2

Aplicada ao **inventário do laudo DRE** (17 itens, seções A-D).

1. Severidade (pré-calibrada)

Cada uma das 17 perguntas tem duas severidades pré-definidas pelo

Responsável Técnico, em AEP_SEVERIDADES_RT (app_cloud.py):

```
```python
AEP_SEVERIDADES_RT = {
"q1": (sev_grau_1_2, sev_grau_3_4),
```

... q1 a q17

```
}
```
```

A coluna usada (Grau de Risco 1-2 ou 3-4) é escolhida automaticamente por `_severidades_por_grau_empresa()`, com base no campo `Grau_Risco` (NR-4) cadastrado para a empresa. Empresas sem Grau de Risco definido usam a coluna 1-2 (default).

2. Probabilidade (contínua)

```
```python
probabilidade = 1 + 3 * percentual_risco # percentual_risco entre 0.0 e 1.0
```

**resultado entre 1,00 e 4,00 (2 decimais)**

```
```
```

- Respostas "Parcial" contam como **0,5** no percentual de risco.
- Respostas "N/A" são **excluídas** do cálculo.
- Setores com menos de `MIN_RESPONDENTES_SETOR` (3) respondentes são agrupados em um conjunto único, para preservar o anonimato na média por setor.

3. Grau de Risco (GR) e Classificação

```
```python
```

```
GR = severidade * probabilidade # 1 decimal, varia de 1,0 a 16,0
```

```
```
```

```
| Condição | Classificação |
```

```
|-----|-----|
```

```
| % de risco > 98% | Crítico (independe do GR — situação insuportável) |
```

```
| GR > 8,0 | Alto |
```

```
| GR > 4,0 (ou % de risco ≥ 70%) | Médio |
```

```
| GR ≤ 4,0 | Baixo |
```

```
| % de risco = 0% | Baixo (GR = severidade, sempre ≤ 4) |
```

Implementado em `_classificar_gr()` (`app_cloud.py`).

4. Coluna "Plano?"

- Plano? = "SIM" a partir da classificação **Médio** (DRPS e DRE).

- No laudo, a seção 6.2 (Medidas de Controle) lista os itens com

Plano? = "SIM"; itens **Alto/Crítico** recebem destaque visual

(fundo #FDF0E7, risco em negrito).

5. Ações de controle recomendadas

ACOES_CONTROLE_RT (`gerar_laudo_aep.py`) é um dicionário

{nº do risco (1-17): [3 ações específicas]}, validado pelo Responsável

Técnico. A seção 6.2 do laudo usa essas ações específicas; para itens

Crítico, uma recomendação genérica de AET/CIPA é exibida em negrito

antes das ações específicas.

6. Cores de classificação (laudo)

```
| Classificação | Cor |
```

```
|-----|-----|
```

```
| Alto | #DC3B24 (laranja/vermelho) |
```

| Crítico | #C0392B (bordô) |

| Médio | #F1C40F (amarelo) |

| Baixo | #5A9F62 (verde) |

■ Fluxos de Dados

Fluxo 1 — Cadastro (manual ou CSV)

...

Admin preenche dados da empresa (CNPJ, Razão Social, período, CNAE, Grau de Risco) + colaboradores (CPF, Função, Departamento)

■

▼

Validação: CPF com 11 dígitos, sem duplicata (CPF + CNPJ)

■

▼

db.salvar_acessos_em_lote(registros) → tabela acessos (upsert)

...

Fluxo 2 — Resposta DRPS (Psicossocial)

...

Colaborador acessa link (?cnpj=XXXX) → informa CPF

■

▼

cpf_hash = SHA-256(CPF)

db.cpf_respondeu(cnpj, cpf_hash)? / status Ativo? / dentro do período?

■ (todas as validações OK)

▼

Exibe 7 blocos COPSOQ III (35 questões, escala 1-5)

■

▼

db.salvar_resposta(dados) → tabela respostas

(médias por dimensão já calculadas e invertidas onde aplicável)

...

Fluxo 3 — Resposta DRE (Ergonômico)

...

Colaborador acessa link (?modulo=aep&cnpj=XXXX) → informa CPF



cpf_hash = SHA-256(CPF)

db.cpf_respondeu_aep(cnpj, cpf_hash)? / status Ativo? / dentro do período?

■ (todas as validações OK)



Exibe 4 seções (A-D, 17 questões) + relatos finais (dor, sugestões)



db.salvar_resposta_aep(dados) → tabela respostas_aep

...

Fluxo 4 — Gerar Laudo PDF (DRPS ou DRE)

...

Admin seleciona empresa → "Gerar Laudo (DRPS/DRE) em PDF"



db.carregar_respostas(cnpj) / db.carregar_respostas_aep(cnpj)



DRPS: calcula médias por dimensão, classifica por BS 8800

DRE: calcula inventário por setor (≥3 respondentes), aplica GR v2



gerar_laudo_pdf() / gerar_laudo_aep_pdf() → PDF (ReportLab)



db.registrar_laudo(cnpj, tipo) → tabela laudos



Download automático do PDF

...

■■ Configuração e Segredos

Segredos do Supabase (`st.secrets`)

db.py lê:

```
```python
url = st.secrets["SUPABASE_URL"]
key = st.secrets["SUPABASE_KEY"]
```
```

- **Local:** arquivo `.streamlit/secrets.toml` (não versionado no Git).
- **Streamlit Cloud:** painel do app → "■" → **Settings** → **Secrets**.

```
```toml
```

## **.streamlit/secrets.toml (exemplo — NÃO use estes valores)**

```
SUPABASE_URL = "https://xxxxxxx.supabase.co"
SUPABASE_KEY = "sua-chave-anon-ou-service-role"
```
```

URLs públicas (constantes em `app\_cloud.py`)

| Constante | Uso |
|-----------|-----|
|-----------|-----|

| | |
|-------------|--|
| ----- ----- | |
|-------------|--|

| | |
|---------|-------------------------------|
| APP_URL | URL do app no Streamlit Cloud |
|---------|-------------------------------|

| | |
|------------------------|--|
| QUEST_PSICOSSOCIAL_URL | Página (GitHub Pages) do questionário DRPS |
|------------------------|--|

| | |
|---------------|---|
| QUEST_AEP_URL | Página (GitHub Pages) do questionário DRE |
|---------------|---|

| | |
|--|------------------------------|
| EB00K_PSICOSSOCIAL_URL / EB00K_AEP_URL | E-books de apoio (módulo RH) |
|--|------------------------------|

As páginas em GitHub Pages (`questionario_*.html`) recebem os parâmetros de query (`?cnpj=...`) e os encaminham para o app Streamlit.

Senha de administrador

- Existe um valor inicial definido em `app_cloud.py` (`SENHA_ADMIN`).
- Ao ser alterada pela primeira vez (opção "Alterar Senha" no módulo Admin), o hash passa a ser armazenado em `config.senha_admin_hash` no Supabase, e **prevalece** sobre o valor inicial do código.
- Senhas de usuários operacionais e de acesso RH (por CNPJ) seguem o mesmo padrão: hash armazenado (`usuarios.senha_hash` / `acessos.senha_rh_hash`), nunca em texto puro.

■ ■ Camada de Banco — `db.py`

| Função | Parâmetros | Retorno / Efeito |

|-----|-----|-----|

| ping() | — | "Desperta" o banco (hibernação do free tier) |

| carregar_acessos() | — | DataFrame da tabela acessos |

| salvar_acessos_em_lote(registros) | lista de dicts | Upsert em acessos |

| atualizar_acesso_campos(cpf, cnpj, campos) | — | Atualiza campos de um colaborador |

| atualizar_acessos_por_cnpj(cnpj, campos) | — | Atualiza todos os colaboradores de uma empresa |

| deletar_acesso_cpf(cpf, cnpj) | — | Remove um colaborador |

| deletar_acessos_empresa(cnpj) | — | Remove todos os colaboradores de uma empresa |

| deletar_todos_acessos() | — | Reset total de acessos |

| carregar_respostas(cnpj) | — | DataFrame de respostas (DRPS) |

| cpf_respondeu(cnpj, cpf_hash) | — | bool |

| salvar_resposta(dados) | dict | Upsert em respostas |

| deletar_respostas_empresa(cnpj) / deletar_todas_respostas() | — | Reset de respostas |

| carregar_respostas_aep(cnpj) | — | DataFrame de respostas_aep (DRE) |

| cpf_respondeu_aep(cnpj, cpf_hash) | — | bool |

| salvar_resposta_aep(dados) | dict | Upsert em respostas_aep |

| deletar_respostas_aep_empresa(cnpj) / deletar_todas_respostas_aep() | — | Reset de respostas_aep |

| listar_cnpjs_com_respostas() | — | Lista de CNPJs com pelo menos uma resposta |

| datas_respostas() / datas_respostas_aep() | — | Listas de datas — usadas no gráfico do módulo Início |

| registrar_laudo(cnpj, tipo) | — | Insere em laudos ("DRPS" ou "DRE") |

| contar_laudos() | — | {"DRPS": n, "DRE": m} |

| carregar_usuarios() / salvar_usuario(registro) / atualizar_usuario_campos(usuario, campos) | — | CRUD de usuarios |

| get_config(chave) / set_config(chave, valor) | — | Leitura/escrita em config |

| exportar_backup_zip() | — | Bytes de um ZIP com CSVs de todas as tabelas |

■ Segurança

Anonimato — hash de CPF

```
```python
cpf_hash = hashlib.sha256(cpf.encode()).hexdigest()
```
```

- Usado como identificador em respostas e respostas_aep.
- Impossível recuperar o CPF original a partir do hash.
- Permite verificar duplicidade de resposta sem armazenar o CPF.

Senhas (admin, operacional, RH)

- Mesmo princípio: `hash_senha( )` aplica SHA-256 antes de gravar.
- `verificar_usuario_operacional( )` compara o hash informado com o armazenado em `usuarios.senha_hash`.
- Senhas de acesso RH são geradas automaticamente (8 caracteres, maiúsculas + dígitos) e exibidas **uma única vez** para a equipe SSTG.

Validação de período

```
```python
hoje = date.today()

data_inicio = ... # acessos.data_inicio_perodo
data_fim = ... # acessos.data_fim_perodo
periodo_ok = data_inicio <= hoje <= data_fim
```
```

Fora do período, o colaborador é bloqueado antes de acessar o questionário.

Boas práticas

- **Nunca** comitar `.streamlit/secrets.toml` com chaves reais no Git.
- O arquivo `.gitignore` do projeto deve excluir `secrets.toml` e qualquer exportação de backup (`*.zip`).

- A documentação (módulo ■ Documentação) é restrita a usuários com `admin_logado=True` (equipe SSTG).

■ Troubleshooting Técnico

`ModuleNotFoundError` (reportlab, plotly, supabase, qrcode, PIL...)

Causa: dependência não instalada no ambiente local.

```
```bash
py -m pip install streamlit pandas reportlab plotly supabase qrcode pillow
```
```

Banco "não responde" / dados não aparecem

Causa: o projeto Supabase free tier hiberna após ~7 dias sem uso.

Solução: o app chama `db.ping()` na inicialização para reativar o banco. Se persistir, abra o painel do Supabase e verifique o status do projeto.

Erro de encoding (emoji) ao depurar via terminal Windows

Causa: o console do Windows usa cp1252 por padrão e não imprime emojis/acentos de algumas strings do app.

Solução: ao depurar via `py -c "...",` use `.encode('ascii', 'replace').decode()` antes de imprimir, ou prefira ler o arquivo diretamente em vez de fazer `print()`.

CSS/HTML personalizado perde `!important`

Causa: `st.markdown(..., unsafe_allow_html=True)` sanitiza atributos `style="..."` inline, removendo `!important`. Como o tema global já define `h1 { color: var(--azul) !important }`, qualquer `<h1>` em HTML customizado herda a cor navy, a menos que uma classe definida no bloco `<style>` injetado sobrescreva.

Solução: sempre estilizar via classes do `<style>` injetado (ex.: `.hero-sstg h1 { color: white !important }`), nunca via `style=""` inline

com !important.

Diagnóstico ao vivo no Streamlit Cloud

Abra <https://<seu-app>.streamlit.app/~/+/> (o iframe do app, no mesmo domínio) e use o DevTools do navegador para inspecionar elementos/estilos diretamente.

■ Dependências

```
'''
```

```
streamlit
```

```
pandas
```

```
reportlab
```

```
plotly
```

```
supabase
```

```
qrcode
```

```
pillow
```

```
'''
```

> **PyMuPDF (fitz)** é usado apenas no módulo ■ Documentação (visualização

> de PDFs página a página) e em scripts de QA — não é necessário para o

> funcionamento principal do sistema.

```
```bash
```

```
py -m pip install streamlit pandas reportlab plotly supabase qrcode pillow
```

```
'''
```

> ■■ Mantenha o `requirements.txt` do repositório (`valter-contador/sstg-e-social`)

> sincronizado com esta lista — é ele que o Streamlit Cloud usa para montar o

> ambiente a cada deploy.

## ■ Próximas Melhorias

- ☐ Acompanhar laudos DRE aplicados no dia a dia e ajustar a metodologia conforme feedback
- ☐ Avaliar exportação de laudos em outros formatos (ex.: Word)
- ☐ Avaliar notificações automáticas de prazo de resposta
- ☐ Completar/regenerar os itens "Documentação de Publicação" e "POP020 Tutorial Telas" no módulo ■ Documentação

